Barbell reveals and resolves demultiplexing and trimming issues in Nanopore data

Rick Beeloo[1,*] **Ragnar Groot Koerkamp**[2] **Xiu Jia**[3] **Marian J. Broekhuizen-Stins**[4] **Lieke van IJken**[4] **Els M. Broens**[4] **Aldert Zomer**[4,†] **and Bas E. Dutilh**[1, 3,*,†]

[1]Department of Biology, Science4Life, Utrecht University, Utrecht, The Netherlands, [2]Department of Computer Science, Karlsruhe Institute of Technology, Karlsruhe, Germany, [3]Institute of Biodiversity, Ecology, and Evolution, Faculty of Biological Sciences, Cluster of Excellence Balance of the Microverse, Friedrich Schiller University Jena, Jena, Germany and [4]Division of Infectious Diseases and Immunology, Utrecht University, Utrecht, The Netherlands

[†]These authors contributed equally to this work.[*]Corresponding authors: Rick Beeloo, beelooinformatics@gmail.com and Bas E. Dutilh, bedutilh@gmail.com

Abstract

Motivation: Oxford Nanopore sequencing enables long-read analysis for diverse applications, but artefacts introduced by Nanopore barcoding are poorly characterised and can compromise demultiplexing accuracy and downstream analyses.

Results: Using a rapid barcoding experiment on 66 diagnostic samples, we found that only 83% of reads followed the expected single-barcode configuration, while 17% showed complex barcode attachments. We observed similar patterns in public datasets, and also in native barcoding datasets where only $30-70\%$ of the reads had barcodes on both ends. Widely used demultiplexers, including Dorado, fail to resolve these cases, leaving approximately 10% of our rapid barcoding reads partially trimmed and contaminated with adapter fragments. We developed Barbell, a pattern-aware demultiplexer that is designed to detect complex barcode configurations. Barbell reduced contaminated reads from >400,000 (Dorado/Flexiplex) to 166 (99.96% reduction), minimised barcode bleeding, and supports custom experimental designs such as dual-end barcodes and shorter barcodes (e.g. Illumina barcodes). We further show that such contamination is widespread in public databases, with Nanopore sequences detected in hundreds of NCBI entries, some of which are responsible for artificial taxonomic connections.

Availability and Implementation: Barbell is open source and available at `https://github.com/rickbeeloo/barbell`.

Key words: Demultiplexing, Barcoding, Sequencing, Reads, Assembly

1. Background

Nanopore sequencing offers real-time, long-read DNA and RNA sequencing with minimal capital investment and laboratory footprint. The R10.4.1 pore architecture with its dual-head design and improved basecalling models increased sequencing accuracy beyond 99% [Kim et al., 2024], enabling applications in 16S amplicon sequencing [Karst et al., 2021], genome assembly [Wick et al., 2023], and metagenomic analysis [Charalampous et al., 2019].

To increase throughput and reduce cost, Nanopore experiments commonly multiplex samples using molecular barcodes introduced during library preparation via tagmentation, ligation, or PCR. In PCR-based barcoding, barcode sequences are incorporated into primer $5'$ ends, tagging each amplicon without additional ligation steps. In tagmentation, transposase fragments DNA while inserting barcoded adapters at cut sites. In ligation, barcoded adapters are enzymatically joined to intact DNA fragment ends. After sequencing, reads are assigned to their source samples through demultiplexing, which involves identifying barcode sequences and often trimming them from read termini.

Despite improved sequencing accuracy, demultiplexing remains error-prone. Barcode bleeding (sample cross-talk or leakage) occurs at rates of 0.056% to 1.5% [Sauvage et al., 2023, Xu et al., 2018, Wu et al., 2021, Holzschuh et al., 2024], primarily from concatenated reads or low-confidence barcode sequences [Xu et al., 2018]. Incomplete adapter trimming further complicates downstream analyses [Liu-Wei et al., 2024, Bonenfant et al., 2022], particularly for low-abundance targets or diagnostic applications [Wu et al., 2021, Holzschuh et al., 2024].

Current demultiplexers face key challenges: Dorado lacks support for custom configurations[1], Splitcode cannot practically

[1] See `https://github.com/nanoporetech/dorado/issues/626`.

© The Author 2025.

handle high indel rates [Sullivan and Pachter, 2024][2], and Flexiplex supports only single-end barcodes [Cheng et al., 2024]. Moreover, none report abnormal barcode attachment patterns, and all assume perfect ligations/tagmentations. These limitations reduce accuracy and restrict custom protocol adoption.

Contributions. We introduce Barbell, an extensive tool for demultiplexing that contributes on several fronts:

- Insight into experimental errors and barcode attachment patterns in Nanopore sequencing
- Handling of complicated custom experimental set-ups (e.g., multiple primers, shorter/longer barcodes (e.g. Illumina barcodes), and dual-end barcodes)
- New barcode scoring scheme reflecting Nanopore errors.
- The option to only include reads displaying safe tagmentation patterns or maximise assignment (e.g., for assembly).

We compared Barbell with existing demultiplexers Dorado and Flexiplex. Our evaluation included trimming errors and the effects of read contamination on taxonomic assignment and genome assembly. We also assessed contamination in NCBI "core nt", overall showing that:

- Only $\approx 80\%$ of our rapid barcoding reads contain the expected single barcode on one side of the read, with the remainder having more complex configurations.
- $\approx 10\%$ of trimmed reads by Dorado and Flexiplex still contain experimental contamination, whereas this is near 0 for Barbell.
- Contamination in trimmed reads propagates into assemblies and produces artificial taxonomic assignments.
- Many sequences in NCBI's database contain Nanopore contamination, including assemblies and amplicons.

2. Methods

2.1. Preliminaries

In this manuscript, we address the problem of demultiplexing, where the goal is to locate a *tag*, denoted by τ, of length $|\tau|$, within a read $R = r_0 \dots r_{n-1}$ of length $n := |R|$. Both τ and R are strings over the alphabet $\{A, C, G, T\}$, extended with IUPAC ambiguity codes (e.g. R, Y, M, N). Let $\sigma := |\Sigma|$ denote the alphabet size.

Each tag τ consists of three parts (or substrings): a left flank F_ℓ, a barcode B, and a right flank F_r. We denote their respective lengths as $|F_\ell|$, $|B|$, and $|F_r|$, such that $\tau := F_\ell \circ B \circ F_r$, where $\circ$ denotes string concatenation. The barcode B, typically 24 bp, comes from a set of g known barcodes, $\beta = \{b_1, b_2, \dots, b_g\}$, whereas F_ℓ and F_r are fixed strings with lengths varying based on the protocol, from $|F_r| = 8$ for native barcoding kits, to, for example, $|F_r| = 50$ for rapid barcoding.

Since all barcodes share the same flanks, we first locate F_ℓ and F_r, and restrict the barcode search to the region between them. We replace B with wildcards (N) to obtain $\tau_N := F_\ell \circ N_{|B|} \circ F_r$, and align this pattern to the read. The aligned N-region defines the substring between the flanks. Due to indels that optimise flank alignment, this substring may be shorter or longer than $|B|$. It is then compared against the barcode set β.

[2] Error patterns up to a defined edit distance are precomputed and stored in a hash map. For long sequences or large edit thresholds, the combinatorial space of possible patterns becomes intractably large.

2.2. Scoring/penalising methods

Throughout the manuscript we use two ways of penalising or scoring sequences. The first measure is the edit distance, which we use to locate τ_N in the read. The second is a subsequence-based scoring function that we use to discriminate between barcodes. This second score is more sensitive to the localised error patterns typical of Nanopore sequencing. Briefly, the subsequence score is derived from the CIGAR string of an edit-distance–based alignment and is designed to reward long, contiguous runs of exact matches. Alignments dominated by uninterrupted matches receive high scores, whereas interleaved patterns of matches and errors receive low scores. A formal definition of the score, together with its motivation and implementation details, is given in Appendix B.

Cut-offs for flank and barcode matching. To locate the flanks (τ_N), Barbell applies automatic cut-offs to distinguish true matches from random alignments. These cut-offs are derived from alignment scores observed in random sequence data of the same effective length, using an empirically fitted lower bound (Figure S3). For barcode assignment, subsequence scores are normalised relative to the maximum achievable (perfect-match) score, and barcodes are accepted based on user-defined minimum score and score-separation thresholds. A full derivation of these cut-offs and their empirical motivation is provided in Appendix C.

2.3. Demultiplexing

Barbell has four main steps: `annotate`, `inspect`, `filter`, and `trim`.

Annotate. In the `annotate` step we locate and score barcodes/flanks in the reads, producing a tsv file with matches per read. In this manuscript we focus on rapid barcoding, but all Nanopore kits are supported (e.g. `SQK-NBD114.96`). The algorithm is described in Algorithm 1. In short, the user supplies a Fasta file (or multiple Fasta files) containing the tag sequences, from which Barbell derives τ_N and β. Barbell then locates τ_N in the reads, extracts the masked region, and compares it to each barcode in β. Whether a barcode matches is based on the subsequence score. By default, the score for b should be $\geq$20% of the perfect score, and the difference between the top two should be $\geq$10%. To get an intuition of these parameters and their effect on the number of demultiplexed reads, see Figure S2. If a barcode is found, this is reported as `Ftag`, where the F denotes front, otherwise the flank is reported as `Fflank` (Figure 1A). In case of dual-end barcodes, the user can provide an additional Fasta file with an `Rtag` (R for rear), of which the incomplete `Rtag` is reported as `Rflank`. For all its searches, Barbell uses Sassy with a default overhang penalty $\alpha = 0.5$, which halves the edit cost for bases that align beyond the read boundary. This allows detection of truncated tags at read ends ([Beeloo and Groot Koerkamp, 2026a]).

Pattern representation. The `annotate` step produces per-read "matches", which Barbell translates into patterns used by both `inspect` and `filter`. A read pattern consists of one or multiple matches, each represented as `<type>[<ori>,<label>,<pos>,<cut>]`, where `<type>` denotes the match class (e.g., `Ftag`, `Rtag`, `Fflank`, `Rflank`), `<ori>` the strand orientation (`fw` or `rc`), `<label>` the barcode label derived from the Fasta header, `<pos>` a location constraint relative to the read end: `@left(0..250)` and `@right(0..250)`, or relative to the previous match: `@prev_left(0..250)`. Here `0..250` indicates a range of 0 to 250 bases and can be tuned to match the expected distances. The `<cut>` indicates the trim direction used in `trim` (`>>` keep after, `<<` keep before). These

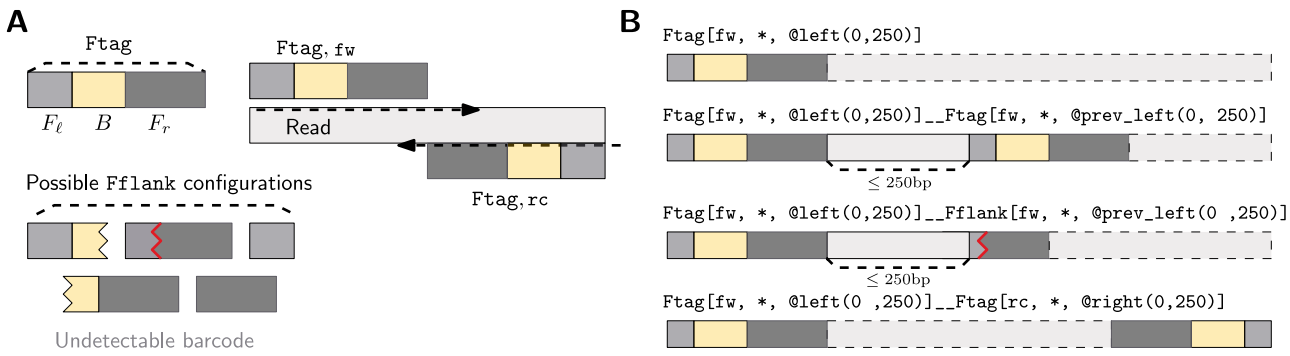


Fig. 1: Example patterns observed in rapid barcoding data. **(A)** shows that an `Ftag` consists of the left flank (F_ℓ), the barcode (B), and the right flank (F_r). If B is undetectable based on the scoring scheme (e.g. absent or bad score) we report it as `Fflank`. If an `Ftag` or `Fflank` matches the user-provided sequence, it is reported as `fw`; if it matches in reverse complement, as `rc`. In **(B)** there are several examples of match patterns observed in rapid barcoding data. The dashed borders (`--`) indicate the part of the read retained after trimming when using the Barbell rapid barcoding maximise preset. By default, a "grouping" of 250 bp is used, as tags are generally shorter than this; however, this value can be modified as a parameter in `inspect`.

Alt text: *Panel A shows a complete Ftag, and fragmented tags called Fflanks. Also it shows that an Ftag on the forward strand is reported as fw, and on the reverse strand as rc. Panel B displays four reads with different Ftag, or partial Ftag matches and the syntax used to describe these patterns.*

match representations are joined to form the patterns depicted in Figure 1B.

Inspect. After assigning a pattern to each read we can count how often patterns are observed to provide an overview of barcode locations and multi-barcode ligation events (Figure 2; bottom).

Filter. In the `filter` step, reads are matched to a pattern and optional trim rules (Figure 1B). These patterns can be copied from the `inspect` overview or manually refined, for example to keep only a specific label. The user can also specify where the read should be cut; for instance, `Ftag[fw,*,@left(0..250),>>]` trims the `Ftag` from the left and keeps the sequence to the right. By default, Barbell only accepts the unambiguous ligation patterns for each kit, but `--maximize` relaxes this filter to retain additional patterns, such as reads with different barcodes on both ends, at the cost of lower assignment confidence. Reads that pass the filter, together with their trim coordinates, are written to a filtered `tsv` file; trimming itself is performed in the `trim` step.

Pattern ambiguity. `inspect` and `filter` serve different purposes. `inspect` shows all patterns detected in the reads, whereas `filter` can be used to extract a subset of reads matching a certain pattern. Here, patterns are not necessarily unambiguous. For example a *tag* may match both `@left(0..250)` and `@right(0..250)` in the case of very short reads. Similarly, a *tag* might be close to the previous tag (`@prev_left(0..250)`) and the right end (`@right(0..250)`). `inspect` always prioritises grouping based on `@prev_left(i..j)` over `@right(i..j)`.

Trim. The `trim` step uses the tsv file from `filter` to trim the reads. For example, `Ftag[fw,*,@left(0..250),>>]` retains the read section after the tag match, and `Ftag[fw,*,@right(0..250),<<]` retains the read section before the tag match. Here, `0..250` only specifies that the tag lies within the first 250 bases; trimming itself is performed based on the exact alignment position of the tag. Tags can be combined for dual-end barcoded reads, for example `Ftag[fw,*,@left(0..250),>>]__Rtag[<<,rc,*,@right(0..250)]` which trims both ends extracting the region between the barcodes (Figure 1B; dashed lines). This step creates an output folder with trimmed reads.

2.4. Further methods

In Appendix A we provide further details on the DNA extraction of the experimental isolates, the genome assemblies, and contamination searching.

2.5. Barbell tool: usage and applications in custom experiments

Barbell is available as a command-line tool, and for standard Nanopore kits (for example, SQK-RBK114-96), a single command automatically identifies flanks and barcodes, sets cut-offs, performs trimming, and generates summary statistics (Figure 2). The tool further accommodates custom experimental designs, including dual-end barcodes and mixed amplicon datasets by allowing users to define their own flanking sequences and barcodes.

3. Results

We developed Barbell to demultiplex Nanopore reads. To obtain experimental data for testing we first performed a Nanopore rapid barcoding experiment (SQK-RBK110.96) where we sequenced 66 diagnostic samples (BC01 to BC66). Then we explored barcode contamination in public data, and how it affects downstream analyses.

3.1. Demultiplexed reads

Sequencing of the 66 diagnostic samples yielded a total of 4,937,349 reads which we demultiplexed with Dorado, Flexiplex, and Barbell. Dorado assigned 4,647,221 (94.1%)[3] to a barcode, Flexiplex 4,667,336 (94.5%), and Barbell 4,224,099 (85.6%). For Barbell we

[3] Dorado unexpectedly retained reads consisting entirely of barcode sequence; we excluded 52,778 such reads for a fair comparison, but users should be aware of this behaviour.

```
Annotating reads...
Auto edit flank cut off: 20
Ftag: 0
GCTTGGGTGTTTAACC-----------------------GTTTTCGCATTTATCGTGAAACGCTTTCGCGTTTTTCGTGCGCCGCTTCA
BC01:           AAGAAAGTTGTCGGTGTCTTTGTG
BC02:           TCGATTCCGTTTGTAGTCGTCTGT
BC03:           GAGTCTTGTGTCCCAGTTACCAGG
BC04:           TTCGGATTCTATCGTGTTTCCCTA
BC05:           CTTGTCCAGGGTTTGTGTAACCTT
...+22 more
⠍ Total: Done: 4937349 records 2m
⠍ Found: Found: 4841102 records 2m
⠍ Missed: Missed: 96247 records 2m

Top 10 most common patterns
Found 759 unique patterns
        Pattern 1: 2333619 occurrences
                Fflank[fw, *, @left(0..250)]
        Pattern 2: 1874215 occurrences
                Ftag[fw, *, @left(0..250)]
        Pattern 3: 320276 occurrences
                Fflank[fw, *, @left(0..250)]__Fflank[fw, *, @prev_left(0..250)]
```

Fig. 2: Barbell command-line interface. Example output when running `barbell kit -kit SQK-RBK114-24 -i reads.fastq -o output`. The interface displays kit information, including whether the `--maximize` option was used (Section 2.3). Inferred flanks (blue), detected barcodes (yellow), and the automatically assigned flank edit-distance cutoff (20 in this example) are shown. The output reports progress at each step and summarises the most frequent sequence patterns in the input Fastq file (3/10 shown), providing a direct overview of double attachments and other experimental artefacts.

Alt text: *Command line output showing read annotation progress with barcode sequences (BC01-BC05 plus 22 more), processing statistics (4.9M records processed, 4.8M matched, 96K missed), and the top 10 most common sequence patterns with occurrence counts and their Ftag and/or Fflank configurations.*

used the `--maximize` option (Section 2.3), and `--use-extended` which detects *fused* barcodes (discussed later in Section 3.2).

The number of demultiplexed reads is a quantitative measure, not necessarily a qualitative one, as we explore in the next sections. The average runtimes were 6 min 50 s for Dorado, 1 min 2 s for Flexiplex, and 4 min 20 s for Barbell. Throughout the following sections we often refer to "patterns" as described in Section 2.3 and Appendix A.1.

3.2. Patterns in rapid barcoding data

Common patterns in reads. Rapid barcoding is designed to attach a single barcode to one end of each read, and in our dataset we observed this expected pattern in 83.0% of reads (4,095,862; Table 1). In total, 680 distinct barcode attachment patterns were detected: 5.3% (263,215 reads) carried barcodes on both ends, 4.3% (213,016) contained two barcodes on the left, and 0.8% (41,128) carried two left barcodes plus a single right-end barcode. Although rare, some reads consisted almost entirely of barcodes, with up to eight in a single read (Figure S5). Overall, ≈17% of reads deviated from the expected design.

To assess whether these deviations are consistent across rapid barcoding datasets, we used Barbell to analyse seven randomly selected rapid datasets from the SRA (Figure 4). Across eight rapid datasets in total (including ours), the expected single barcode pattern was observed in 89% of reads on average, followed by 3% with barcodes on both ends and 2.3% with two barcodes on the left.

For context, we also analysed eight randomly selected native barcoding datasets. While not the focus of this study, these show a markedly different distribution, with the expected barcode pair on both ends observed in 50.5% of reads, followed by 41% with a barcode only on the left and 6.3% only on the right (Figure 4). The remainder had different configurations (Figure 4B). Note that "left" and "right" are defined by the input read orientation and may not reflect true barcode attachment bias.

Incorrectly trimmed reads. To detect residual adapter and barcode sequences after trimming, we scanned all trimmed reads for flanks and barcodes with Sassy [Beeloo and Groot Koerkamp, 2026a,b] using an edit-distance-based search (see Appendix A). Nanopore adapter and barcode remnants (hereafter "contamination") were detected in 10.0% of reads trimmed by Dorado (464,518), 8.7% by Flexiplex (406,450), and 0.004% by Barbell (166). The few contaminated reads detected after Barbell trimming arose from Sassy's overhang-based search, which assigns lower cost to missing terminal bases and can therefore detect a second partial barcode only after trimming the first.

Short reads ($\leq$ 250 bp; 880,637 total) were trimmed inconsistently across tools. Dorado retained 88.1% (775,409) of these reads after trimming, Flexiplex 89.0% (783,496), and Barbell 43.5% (383,320). Here, "retained" indicates reads not entirely consumed by adapter/barcode removal. Among retained short reads, contamination was found in 44.5% (345,142) of Dorado-trimmed reads, 40.3% (315,877) of Flexiplex-trimmed reads, and 0.04% (160) of Barbell-trimmed reads. In Dorado and Flexiplex, contamination was primarily associated with failure to detect additional barcodes when multiple were present (Figure 3A). These patterns produced characteristic length peaks at ∼60 bp (one remaining barcode) and ∼120 bp (two remaining barcodes). An entire flank with barcode is 90 bp, so the remaining copy was shorter by ∼30 bp as its prefix was often partially lost in such double barcode cases (Figure S6).

Longer reads (>250 bp) were also trimmed inconsistently. Relative to Barbell, 424,207 reads trimmed by Dorado and 403,233 reads trimmed by Flexiplex were longer for the same input reads, with length differences exceeding 10 bp in 108,095 and 80,659 reads, respectively. Of these, 93.1% (Dorado) and 91.7% (Flexiplex) contained contamination (Figure 3B). Even among reads $\geq$1,000 bp, typically used for genome assembly, contamination was detected in 64,157 Dorado reads, 43,705 Flexiplex reads, and 3 Barbell reads. Overall, incorrect trimming affected ≈ 10% of reads when using Dorado or Flexiplex, mostly short reads, but persisted in long reads.

Incorrect trimming and taxonomic assignment. Next, we evaluated how contamination affected the taxonomic assignment of reads. Since Flexiplex and Dorado showed similar trimming patterns and contamination rates (Figure 3), we focus on Dorado comparisons for conciseness. It is the most widely adopted tool and is integrated in MinKNOW workflows. The reads trimmed by Dorado and Barbell were annotated using Centrifuger, which assigns taxonomy based on k-mer matches between reads and a reference database (e.g. RefSeq [Tatusova et al., 2013]). When a read matches multiple reference entries belonging to different taxa, Centrifuger reports the lowest common ancestor (LCA) shared across all matches in the taxonomic hierarchy. As a result, reads with heterogeneous species-level matches may be assigned only at higher ranks (e.g. family level), without an associated specific RefSeq accession.

We compared RefSeq-based classifications after trimming with Dorado and Barbell. Of the 4,647,221 Dorado reads, 3,808,962 (82.0%) received a taxonomic assignment, compared to 3,588,733 of 4,224,099 Barbell reads (85.0%). At the family level, 3,370,802 Dorado reads and 3,194,077 Barbell reads were resolved. Across the 3,442,152 reads for which either tool assigned a family,

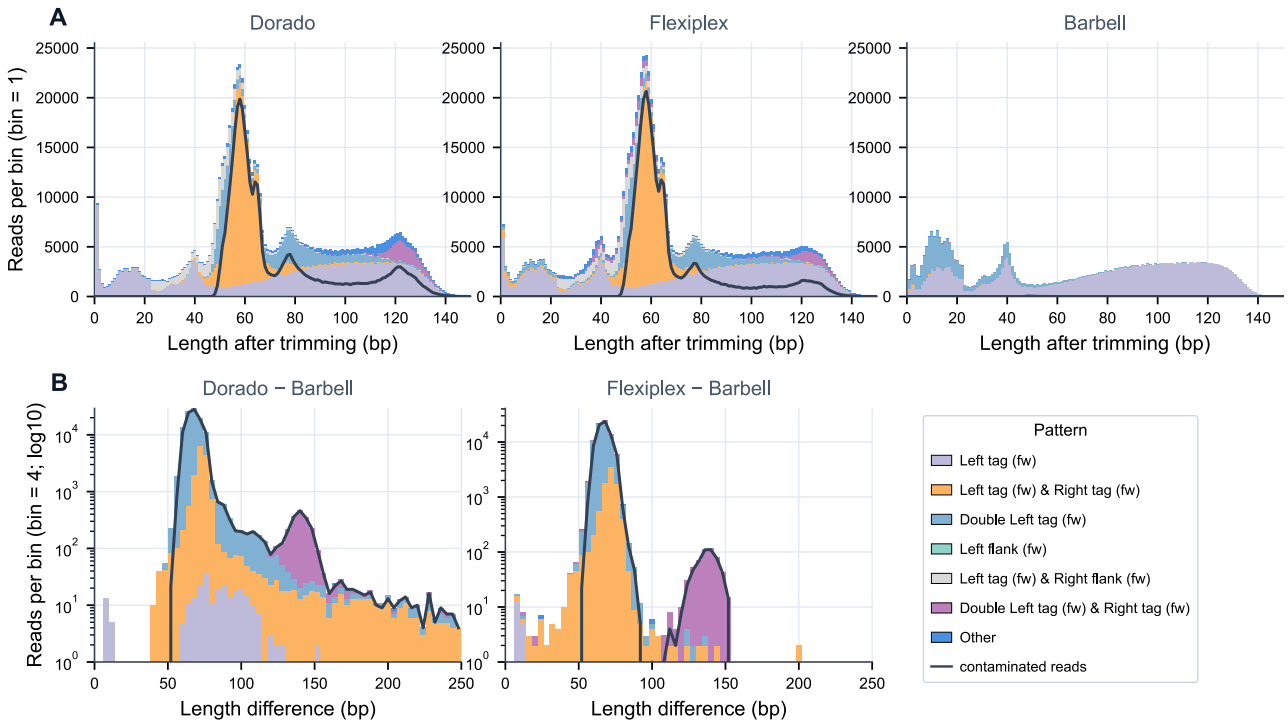


Fig. 3: Trimmed read length comparisons. (A) Distribution of read lengths after trimming short reads (≤250 bp) for Dorado, Flexiplex, and Barbell. Bars are coloured according to the pattern assigned by Barbell, as outlined in Table 1. Most reads were trimmed from 250 bp to ≤150 bp (280 reads >150 bp not shown). Dorado and Flexiplex produced many trimmed reads of ≈60 bp, visible as a prominent peak, which were absent in Barbell output. These ≈60 bp trimmed reads originated from sequences containing two adjacent barcodes without sequence in between: Dorado and Flexiplex recognised only one barcode and output the remaining barcode sequence as a valid read, whereas Barbell detected both barcodes and removed the entire read as contamination. The black line indicates trimmed reads that contained detectable Nanopore adapter sequences (see Methods), which closely tracked the ≈60 bp contamination peak, confirming these were artefact sequences rather than genuine biological reads. **(B)** Trimmed read length differences for long reads >250 bp comparing reads trimmed by Dorado vs. Barbell (left) and Flexiplex vs. Barbell (right). Note the logarithmic y-axis. Both Dorado and Flexiplex output longer trimmed reads than Barbell, often corresponding to an additional single undetected (≈60 bp peak) or two undetected (≈120 bp peak) barcodes per read, similar to those in **(A)**. As the difference between the tools was generally ≤10 bp, we only showed differences >10 bp.

Alt text: *Two panels of histograms. Panel A (linear y-axis, 0-25000 reads per bin) shows trimmed read length distributions up to 140 bp for Dorado, Flexiplex, and Barbell, stacked and coloured by barcode pattern. Dorado and Flexiplex each have a sharp peak at 60 bp overlaid by a black line representing contamination; this peak is absent in Barbell. Panel B (log10 y-axis) shows length differences for longer reads comparing Dorado vs. Barbell and Flexiplex vs. Barbell, with peaks at 60 bp and 120 bp.*

3,107,631 (90.3%) were assigned to the same family and 334,521 (9.7%) to different families. Most discrepancies arose from Barbell-trimmed reads that were unclassified at the family level, whereas the corresponding Dorado-trimmed reads were assigned to *Enterobacteriaceae* (203,122; 60.7%), including many classified as *E. coli* (48,391; 23.8%).

Among reads assigned to *Enterobacteriaceae*, 176,359 (86.8%) produced multiple RefSeq matches spanning different species, leading Centrifuger to report only the family-level LCA, as described above, without a specific RefSeq accession. The remaining 26,763 reads (13.2%) were assigned to 244 unique RefSeq entries. Analysis of the assigned reads showed that residual *Mu* transposon sequence from Nanopore Rapid Barcode flanks matched endogenous *Mu*-like elements in bacterial genomes: 24,857 (92.9%) of alignments mapped to regions characteristic of transposon- and *Mu* phage integration sites (Appendix D), a result confirmed by direct alignment to the *Mu* phage genome. Thus, untrimmed tagmentation-derived *Mu* sequences created artificial matches to endogenous *Mu*-like elements, producing misleading taxonomic assignments.

The Genome Taxonomy Database (GTDB) is often used for taxonomic annotation as its high-quality sequences are expected to yield accurate assignments. Using Centrifuger with GTDB, 3,705,248 reads were annotated by either tool, of which 346,016 (9.3%) were assigned to a different family. Where Barbell's reads were unassigned, Dorado's reads were mostly assigned to *Balneolaceae* (103,547; 29.9%) and *Streptomycetaceae* (98,098; 28.3%). These assignments originated from read contamination matching contamination in public assemblies (see Section 3.4). Specifically, 67.4% of *Streptomycetaceae*, all *Streptomyces* species, were assembled by Jørgensen et al. [2024]. All *Balneolaceae* were from a single *Gracilimonas* assembly (`GCF_040117685.1`) by Lim et al. [2025]. Notably, the matched regions included both *Mu* transposon flanks and Nanopore barcode sequences, making

Table 1. Top 10 barbell patterns in rapid barcoding. This table shows the 10 most common out of 680 total patterns detected in the reads. The first column contains the colour used in the plots throughout this manuscript. The "Pattern" column shows the read pattern assigned by Barbell (see Section 2.3). The "All" column is based on the Barbell `annotate` output, and the other columns show the number of reads with the pattern assigned by Barbell that were trimmed and output by Dorado, Flexiplex, and Barbell. The percentages are based on the total number of demultiplexed reads by each tool (Total). The discrepancy between the "All" count and the count in the "Barbell" column corresponds to the number of cases where the full read sequence was trimmed, e.g. when the entire read consisted of barcodes or flanks. The `None` pattern indicates that Barbell did not find any barcodes and thus no pattern was assigned to those reads.

Pattern	Dorado	Flexiplex	Barbell	All
`Ftag[fw,*,@left(0..250)]`	3,971,269 (85.5%)	4,010,601 (85.9%)	4,029,358 (95.4%)	4,095,862 (83.0%)
`Ftag[fw,*,@left(0..250)]__Ftag[fw,*,@right(0..250)]`	258,785 (5.6%)	259,800 (5.6%)	17,251 (0.4%)	263,215 (5.3%)
`Ftag[fw,*,@left(0..250)]__Ftag[fw,*,@prev_left(0..250)]`	195,919 (4.2%)	207,737 (4.5%)	175,521 (4.2%)	213,016 (4.3%)
`Fflank[fw,*,@left(0..250)]`	7,939 (0.2%)	28,783 (0.6%)	0 (0.0%)	94,685 (1.9%)
`None`	13,706 (0.3%)	0 (0.0%)	0 (0.0%)	91,428 (1.9%)
`Ftag[fw,*,@left(0..250)]__Fflank[fw,*,@right(0..250)]`	51,251 (1.1%)	51,413 (1.1%)	0 (0.0%)	52,948 (1.1%)
`Ftag[fw,*,@left(0..250)]__Ftag[fw,*,@prev_left(0..250)]__Ftag[fw,*,@right(0..250)]`	37,866 (0.8%)	40,293 (0.9%)	1,901 (0.0%)	41,128 (0.8%)
`Ftag[fw,*,@left(0..250)]__Fflank[fw,*,@prev_left(0..250)]`	17,962 (0.4%)	17,378 (0.4%)	0 (0.0%)	22,141 (0.4%)
`Ftag[fw,*,@left(0..250)]__Ftag[fw,*,@prev_left(0..250)]__Ftag[fw,*,@prev_left(0..250)]`	8,638 (0.2%)	12,785 (0.3%)	12 (0.0%)	13,178 (0.3%)
`Ftag[fw,*,@left(0..250)]__Ftag[fw,*,@prev_left(0..250)]__Fflank[fw,*,@right(0..250)]`	11,894 (0.3%)	12,655 (0.3%)	0 (0.0%)	13,082 (0.3%)
Total	4,647,221	4,667,336	4,224,099	4,937,349

an endogenous *Mu*-like origin implausible. Because the rapid barcoding region spans only 90 bp (see Methods), restricting taxonomic assignments to matches $\geq$ 100 bp reduced rapid barcoding contamination, lowering family-level discrepancies to 129 reads at the cost of a $\approx 25\%$ reduction in assigned reads.

Double barcode attachment and bleeding. Double left-end barcodes were detected in 213,016 reads (4.3% of total; Table 1). In 100,042 (47.0%) of these, the right flank of the first barcode was directly fused to the second barcode, resulting in loss of the left flank and partial deletion of the second barcode (Figure S6). Consequently, the mean edit distance to the second barcode was higher than to the first (7 vs. 3). Fusion-associated deletions varied by barcode and were also observed in public datasets (Appendix E). Barbell users can enable fusion detection using the `--use-extended` option.

Of the reads with double barcodes, 212,048 (99.5%) carried the same barcode twice and therefore did not affect demultiplexing, although being the main source of trimming errors. In the remaining 968 reads (0.5%), the tools disagreed: Dorado selected the outer barcode copy, whereas Barbell selected the inner copy under the assumption that the first tagmentation was the correct one. Read-level taxonomic annotation supported the Barbell assignment in 53.2% of cases, compared to 4.4% support for the Dorado (outer-barcode) assignment.

Propagation into assemblies. Assemblies were successfully generated for 64 of 66 samples; the remaining two contained too few reads for assembly. Across these 64 assemblies (59 bacterial and 5 fungal; see Table S1), no systematic differences in continuity were detected between assemblies from Barbell and Dorado trimmed reads (see Supplementary Section C.1; paired Wilcoxon signed-rank test, $p = 0.22$). For the bacterial subset, CheckM2 reported near-identical quality metrics: Dorado assemblies had mean completeness of 99.34% and mean contamination of 1.00%, whereas Barbell assemblies had mean completeness of 99.35% and mean contamination of 1.00%.

However, CheckM2 estimates contamination using single-copy marker genes and can therefore fail to detect non-biological, experiment-specific artefacts originating from library preparation. To identify these artefacts, we screened all 64 assemblies for residual rapid-barcoding sequence. Contamination was detected in seven assemblies from Dorado-trimmed reads, and in none of the assemblies produced from Barbell-trimmed reads. For example, a contig from a *Saccharomyces cerevisiae* assembly contained a 92-bp segment that did not occur in canonical *S. cerevisiae* genomes but instead matched diverse bacterial and synthetic sequences — a leftover produced when Dorado trimmed only the first of two barcode occurrences (Figure S7). Thus, although marker-based completeness/contamination metrics were similar, Barbell effectively removed experimental artefacts that can confound taxonomic annotation.

3.3. Comparing scoring schemes

Overall, Dorado demultiplexed 456,953 more reads than Barbell. Of these, 92% (428,823) were reads that Barbell had detected but excluded from its final output because they failed to match expected rapid-barcoding patterns or were trimmed to empty sequences.

To assess the accuracy of these additional assignments, we compared species-level taxonomic annotations of trimmed reads with those of the assemblies associated with their assigned barcodes. Only 30,308 (6.6%) of the additional Dorado-demultiplexed reads were taxonomically consistent, indicating that most were incorrectly assigned. Of those consistent with taxonomy, 21,645 were excluded by Barbell due to conservative scoring. This included 13,706 reads whose barcoding flanks exceeded the default edit-distance cutoff and 7,939 reads annotated as `Fflank` due to inconclusive subsequence scoring.

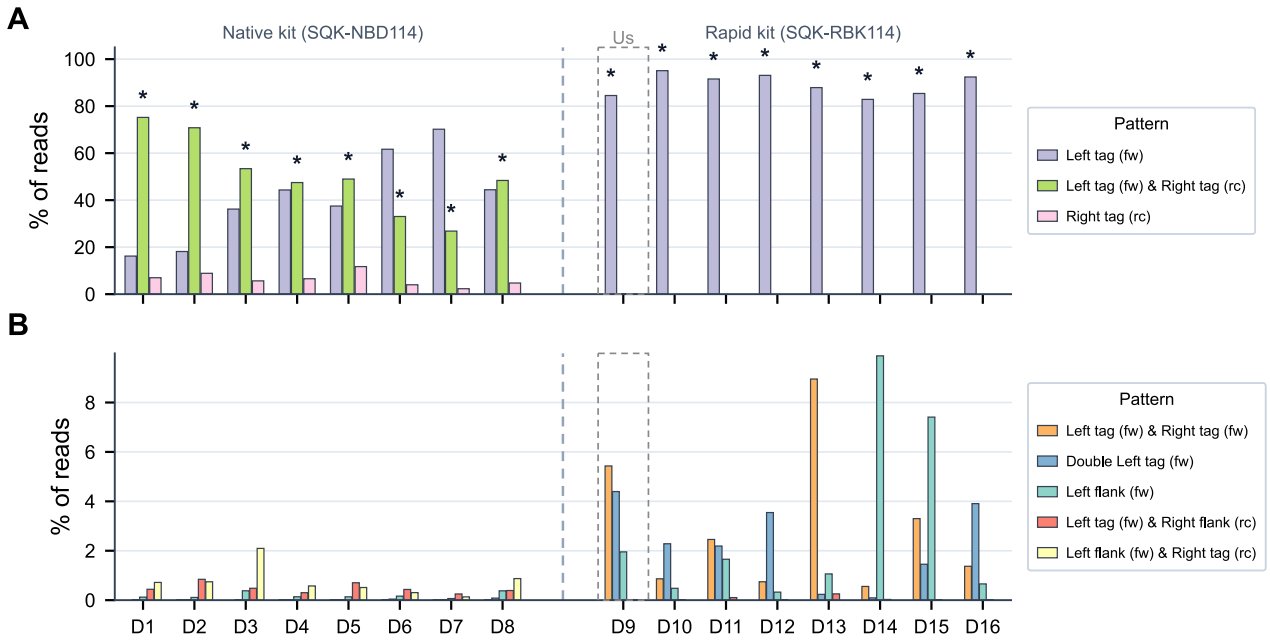


Fig. 4: Barcode patterns in public native and rapid barcoding datasets. Barcode attachment patterns across eight rapid barcoding datasets (SQK-RBK114; including ours, dashed rectangle) and eight native barcoding datasets (SQK-NBD114) from the SRA. All datasets were analysed with Barbell using `--maximize` (and `-use-extended` for rapid kits (Sections 2.3 and 3.2)). Panel **A** shows the three most frequent patterns per dataset ($> 10\%$ of reads), while **B** shows less frequent patterns (at most 10% of reads). The asterisk (∗) above a bar indicates the expected pattern for the given kit. SRA accession numbers for each dataset (`Dx`) are listed in Table S2.

Alt text: *Two panels of bar charts showing barcode attachment pattern frequencies across 16 datasets: D1-D8 native kit SQK-NBD114, D9-D16 rapid kit SQK-RBK114. Panel A (y-axis 0-100%): Three patterns per dataset: "Left tag (fw)" (purple), "Left tag (fw) and Right tag (rc)" (green), "Right tag (rc)" (pink). Rapid kit datasets are dominated by the expected pattern (>80-90% of reads), while native kit datasets show more mixed distributions, rarely exceeding 60-70% for any single pattern. Panel B (y-axis 0-8%): Five rarer patterns shown. Native kit bars generally stay within 0-2%, while some rapid kit datasets reach up to around 9%.*

Conversely, Barbell demultiplexed 86,609 reads that Dorado failed to assign. Of these, 60,009 (69.3%) were taxonomically consistent with their corresponding assemblies, with the remaining reads being taxonomically unclassified.

Note that the comparison in this section focuses on the quantitative aspect of demultiplexing (i.e., the number of assigned reads), whereas the earlier trimming analysis (Section 3.2) evaluated qualitative accuracy (i.e., removal of contamination). Overall, Barbell recovered 60,009 taxonomically consistent reads that Dorado failed to assign while missing 30,308 reads that Dorado likely demultiplexed correctly. However, as shown previously, 10% of Dorado-demultiplexed reads retained barcode contamination, compared to 0.004% for Barbell, demonstrating that high assignment rates do not necessarily reflect higher quality.

3.4. Barcodes and their flanks in public databases

Adapter contamination in genome assemblies has previously been reported for Illumina data [Moeller et al., 2024]. Because we observed that Nanopore sequences are not always removed by standard tools such as Dorado, we queried the NCBI "core nucleotide" database (≈ 810 GB) for Nanopore barcode and flank sequences.

Overview. We found detectable footprints of both rapid and native Nanopore barcoding kits across viruses, bacteriophages, bacteria, plasmids, organelles, and rRNA records. Filtered match tables have been uploaded to Zenodo.[4] Read-level re-analyses (read counts, trimming parameters, and assembly commands) are provided in Appendix G and summarised below.

Rapid barcodes. For rapid kits we observed 103 matches where both rapid barcode flanks and barcode sequences were present, including 68 exact (0-edit) matches across 67 assemblies; additional hits to flanks alone were excluded because they may reflect endogenous *Mu* transposons (Section 3.2). The most striking example was *Photobacterium leiognathi* SV5.1 (`CP131573.1`), where BC86 occurred 11 times across a 1.43 Mb contig. The barcodes were scattered internally, consistent with scaffolding of smaller contigs separated by `N` stretches. Repeating the demultiplexing and trimming of the raw reads using Barbell (see Appendix G) followed by reassembly removed barcode sequences and produced contigs consistent with the expected chromosomal arrangement. Another clear case was an *E. coli* plasmid (`CP165501.1`) carrying BC10 and BC09 overhangs at

[4] Filtered match results can be found at `https://zenodo.org/records/17396505`.

opposite ends of the contig. The sample was annotated as a BC10 sample, but Barbell revealed barcode bleeding with 13.1% of the raw reads from the BC09 sample. Re-assembly from Barbell reads for just BC10 eliminated the barcode contamination. Other rapid-kit examples included plasmids, mobile elements, and assemblies from Jørgensen et al. [2024], *Streptococcus thermophilus* (`CP072431.1`), and *Staphylococcus aureus* (`CP150769`).

Native barcodes. For native kits we detected 462 matches to native flanks and barcodes, of which 270 were exact matches across 284 assemblies. Native-kit contamination was widespread and spanned viruses (including human SARS-CoV-2; `OV192362.1`), bacteriophages (`OP583592`, `OR487170.1`, `PP989835.1`), bacteria (e.g., *Mycolicibacterium novocastrense*, `CP097264.1` — 50 matches to NB02), parasites, fungi, short rRNA records, and organellar genomes (mitochondrial *Tonna galea*, `NC_082277`; chloroplast *Cephaleuros karstenii*, `NC_060534`). For example, an 8,423 bp plasmid (`CP142556.1`) contained a remnant NB13 barcode at the contig end. Despite being annotated as circular, self-alignment showed an overlap leaving the barcode as an overhang, and subsequent Illumina polishing by the authors did not remove the artefact [Li et al., 2024], demonstrating that circularity calls alone do not guarantee contamination-free sequences.

Thus, both rapid and native Nanopore barcoding kits have left detectable, and sometimes assembly-affecting, footprints in public sequence databases. Careful read-level trimming and reassembly removed barcode-derived artefacts.

4. Discussion

Barbell is a powerful demultiplexing tool that provides insight into the adapter and barcode patterns in Nanopore reads, many of which may result from experimental artefacts. It substantially reduced trimming errors and minimised barcode bleeding.

Barcode patterns. A large portion of research on Nanopore sequencing focuses on establishing error rates and mitigating their effects in downstream analysis by generating consensus sequences [Rang et al., 2018, Greenfield et al., 2014, Liu et al., 2024]. However, much less attention has been paid to what happens to reads during the experimental steps such as tagmentation and ligation. Only ≈80% of reads in our rapid barcoding experiments were of the expected configuration, with other public experiments showing similar patterns (average 89%). The remaining ≈20% contained multiple barcodes in different configurations.

Specifically, reads having two barcodes on the left or barcodes on both ends of the read are problematic for existing tools (Figure 3). For double-left barcodes, the second copy often lacked the entire left flank and a partial prefix of the second barcode (Figure S6). As for the physicochemical mechanism underlying these sequence anomalies, we did not observe any abnormal spikes in the pore signal that would indicate secondary structure (Appendix F). An alternative hypothesis is that there are biases in Nanopore sequencing related to the *Mu* target site [Chen et al., 2025]. However, we did not conclusively identify a signature explaining this phenomenon, and further investigation is warranted. Such fusions are difficult to demultiplex for two reasons. First, the entire loss of the left flank makes it difficult to locate the barcode region in the first place, and second, prefix loss of the barcode increases its edit distance to the correct barcode and potentially lowers it to other barcodes. To address this, we added the fusion pattern to Barbell (enabled by `--use-extended`).

While identical double barcodes on the left side heavily impaired trimming by Dorado, around 0.5% of the double left reads had two different barcodes. Dorado's selection of the outer barcode here is likely a source of barcode bleeding based on the taxonomy analysis.

Contamination. Contamination from reagents and kits is well known to affect downstream analyses [Salter et al., 2014]. Many metagenomes in the MGnify database contain Illumina adapter contamination [Moeller et al., 2024], and we demonstrated that Nanopore barcodes and their flanking sequences can also appear in assemblies when using existing demultiplexers. Moreover, our analysis of the "core nucleotide" database suggests that many public assemblies contain Nanopore contamination. A valuable next analysis would be to repeat such analyses for other public resources such as GTDB, which is commonly used for taxonomic assignment. The presence of such contamination can generate spurious taxonomic signals, particularly when untrimmed barcodes and flanks from demultiplexers form misleading links with contamination in the public databases (Section 3.2). Therefore, researchers should exercise caution when using public data as reference material. Special attention should be paid to whether matches stem from barcodes or flanking regions. Stricter post-processing rules—for example, requiring matches of at least 100 bp—can help reduce such spurious matches.

Related to the public database analysis (Appendix D), we note that the rapid kit (SQK-RBK114-96) uses *Mu* transposon sequences to tagment barcodes. Since these *Mu* transposons naturally occur in several *Enterobacteriaceae* species, they can yield apparent "false positive" `Fflank` matches; conversely, residual flanking sequence may be annotated as a *Mu* transposon. Although such reads can still be included by allowing `Fflank` matches within reads, it is important to be aware of these potential effects.

Researchers are increasingly developing custom experiments using their own barcodes, primers, or other tags, often resorting to custom demultiplexing scripts. Barbell is specifically designed for such cases as a modular tool, where the flank and barcode sequences can be changed to fit the user's needs without requiring users to write their own scripts or code. This functionality supports experimental design, helps detect potential issues, and enables subsetting of reads based on expected patterns.

5. Conclusion

We demonstrated that commonly used demultiplexers leave approximately 10% of Nanopore reads improperly trimmed in our dataset, which can significantly impact downstream analyses such as taxonomic annotation and genome assembly. These effects are further exacerbated by the presence of similar contamination in public databases, including the core nucleotide database and GTDB, which can create artificial connections lacking true biological meaning. Barbell detected complex barcode patterns and reduced barcode bleeding and trimming errors.

5.1. Availability of data and materials

Generated reads are available under BioProject `PRJEB100828`. All code, reproducible analysis steps, and public data search results are available on Zenodo (`https://doi.org/10.5281/zenodo.19494282`, `https://doi.org/10.5281/zenodo.17396505`). The Barbell version used to generate the results is also available on Zenodo (`https://doi.org/10.5281/zenodo.19493956`).

5.2. Funding

This work was supported by ZonMW project 541003001, the European Research Council (ERC) Consolidator grant 865694: DiversiPHI, Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy – EXC 2051 – Project-ID 390713860, and the Alexander von Humboldt Foundation in the context of an Alexander von Humboldt-Professorship funded by the German Federal Ministry of Education and Research.

5.3. Author contributions statement

RB implemented the tool, conducted tests, and wrote the manuscript. RGK helped with the search algorithm (sassy) and provided valuable insights for the tool's implementation. BED and AZ provided guidance for the experiments and analyses. XJ and AZ tested Barbell on several datasets. LvIJ, EB and MBS sequenced isolates. All authors provided feedback on the manuscript.

5.4. Acknowledgments

We thank Torsten Schubert and Swapnil Doijad from the Viral Ecology and Omics (VEO) Group, Cluster of Excellence Balance of the Microverse, for providing Nanopore chemistry context and noticing the issue with Dorado's trimming, respectively.

References

R. Beeloo and R. Groot Koerkamp. Sassy: Fuzzy searching dna sequences using simd. *Bioinformatics*, 2026a. doi: 10.1093/bioinformatics/btag244.

R. Beeloo and R. Groot Koerkamp. Sassy2: Batch searching of short dna patterns. *bioRxiv*, 2026b. doi: 10.64898/2026.03.10.710811.

Q. Bonenfant, L. Noé, and H. Touzet. Porechop_abi: discovering unknown adapters in oxford nanopore technology sequencing reads for downstream trimming. *Bioinformatics Advances*, 3(1): vbac085, 2022. ISSN 2635-0041. doi: 10.1093/bioadv/vbac085.

T. Charalampous, G. L. Kay, H. Richardson, A. Aydin, R. Baldan, C. Jeanes, D. Rae, S. Grundy, D. J. Turner, J. Wain, R. M. Leggett, D. M. Livermore, and J. O'Grady. Nanopore metagenomics enables rapid clinical diagnosis of bacterial lower respiratory infection. *Nature Biotechnology*, 37(7):783–792, 2019. ISSN 1546-1696. doi: 10.1038/s41587-019-0156-5.

Z. Chen, C. T. Ong, L. T. Nguyen, H. J. Lamb, O. González-Recio, M. Gutiérrez-Rivas, S. J. Meale, and E. M. Ross. Biases from oxford nanopore library preparation kits and their effects on microbiome and genome analysis. *BMC Genomics*, 26(1): 504, 2025. ISSN 1471-2164. doi: 10.1186/s12864-025-11649-z.

O. Cheng, M. H. Ling, C. Wang, S. Wu, M. E. Ritchie, J. Göke, N. Amin, and N. M. Davidson. Flexiplex: a versatile demultiplexer and search tool for omics data. *Bioinformatics*, 40(3):btae102, 2024. ISSN 1367-4811. doi: 10.1093/bioinformatics/btae102.

C. Delahaye and J. Nicolas. Sequencing dna with nanopores: Troubles and biases. *PLOS ONE*, 16(10):e0257521, 2021. ISSN 1932-6203. doi: 10.1371/journal.pone.0257521.

V. Di Pilato, C. Bonaiuto, F. Morecchiato, A. Antonelli, T. Giani, and G. M. Rossolini. Next-generation diagnostics of bloodstream infections enabled by rapid whole-genome sequencing of bacterial cells purified from blood cultures. *eBioMedicine*, 114:105633, 2025. ISSN 2352-3964. doi: 10.1016/j.ebiom.2025.105633.

H. Gamaarachchi, H. Samarakoon, S. P. Jenner, J. M. Ferguson, T. G. Amos, J. M. Hammond, H. Saadat, M. A. Smith, S. Parameswaran, and I. W. Deveson. Fast nanopore sequencing data analysis with slow5. *Nature Biotechnology*, 40(7): 1026–1029, 2022. ISSN 1546-1696. doi: 10.1038/s41587-021-01147-4.

E. Garrison and A. Guarracino. Unbiased pangenome graphs. *Bioinformatics*, 39(1):btac743, 2022. ISSN 1367-4811. doi: 10.1093/bioinformatics/btac743.

P. Greenfield, K. Duesing, A. Papanicolaou, and D. C. Bauer. Blue: correcting sequencing errors using consensus and context. *Bioinformatics*, 30(19):2723–2732, 2014. ISSN 1367-4803. doi: 10.1093/bioinformatics/btu368.

A. Holzschuh, A. Lerch, B. S. Fakih, S. M. Aliy, M. H. Ali, M. A. Ali, D. J. Bruzzese, J. Yukich, M. W. Hetzel, and C. Koepfli. Using a mobile nanopore sequencing lab for end-to-end genomic surveillance of plasmodium falciparum: A feasibility study. *PLOS Global Public Health*, 4(2):e0002743, 2024. ISSN 2767-3375. doi: 10.1371/journal.pgph.0002743.

J. Huerta-Cepas, F. Serra, and P. Bork. Ete 3: Reconstruction, analysis, and visualization of phylogenomic data. *Molecular Biology and Evolution*, 33(6):1635–1638, 2016. ISSN 1537-1719. doi: 10.1093/molbev/msw046.

T. S. Jørgensen, O. S. Mohite, E. B. Sterndorff, M. Alvarez-Arevalo, K. Blin, T. J. Booth, P. Charusanti, D. Faurdal, T. Ø. Hansen, M. Nuhamunada, A.-S. Mourched, B. Ø. Palsson, and T. Weber. A treasure trove of 1034 actinomycete genomes. *Nucleic Acids Research*, 52(13):7487–7503, 2024. ISSN 1362-4962. doi: 10.1093/nar/gkae523.

S. M. Karst, R. M. Ziels, R. H. Kirkegaard, E. A. Sørensen, D. McDonald, Q. Zhu, R. Knight, and M. Albertsen. High-accuracy long-read amplicon sequences using unique molecular identifiers with nanopore or pacbio sequencing. *Nature Methods*, 18(2):165–169, 2021. ISSN 1548-7105. doi: 10.1038/s41592-020-01041-y.

B. Y. Kim, H. R. Gellert, S. H. Church, A. Suvorov, S. S. Anderson, O. Barmina, S. G. Beskid, A. A. Comeault, K. N. Crown, S. E. Diamond, S. Dorus, T. Fujichika, J. A. Hemker, J. Hrcek, M. Kankare, T. Katoh, K. N. Magnacca, R. A. Martin, T. Matsunaga, M. J. Medeiros, D. E. Miller, S. Pitnick, M. Schiffer, S. Simoni, T. E. Steenwinkel, Z. A. Syed, A. Takahashi, K. H.-C. Wei, T. Yokoyama, M. B. Eisen, A. Kopp, D. Matute, D. J. Obbard, P. M. O'Grady, D. K. Price, M. J. Toda, T. Werner, and D. A. Petrov. Single-fly genome assemblies fill major phylogenomic gaps across the drosophilidae tree of life. *PLOS Biology*, 22(7):e3002697, 2024. ISSN 1545-7885. doi: 10.1371/journal.pbio.3002697.

M. Kolmogorov, J. Yuan, Y. Lin, and P. A. Pevzner. Assembly of long, error-prone reads using repeat graphs. *Nature Biotechnology*, 37(5):540–546, 2019. ISSN 1546-1696. doi: 10.1038/s41587-019-0072-8.

H. Li. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18):3094–3100, 2018. ISSN 1367-4811. doi: 10.1093/bioinformatics/bty191.

H. Li, B. Handsaker, A. Wysoker, T. Fennell, J. Ruan, N. Homer, G. Marth, G. Abecasis, and R. Durbin. The sequence alignment/map format and samtools. *Bioinformatics*, 25(16): 2078–2079, 2009. ISSN 1367-4803. doi: 10.1093/bioinformatics

/btp352.

X. Li, H. Hu, Y. Zhu, T. Wang, Y. Lu, X. Wang, Z. Peng, M. Sun, H. Chen, J. Zheng, and C. Tan. Population structure and antibiotic resistance of swine extraintestinal pathogenic escherichia coli from china. *Nature Communications*, 15(1): 5811, 2024. ISSN 2041-1723. doi: 10.1038/s41467-024-50268-2.

S. J. Lim, O. Natarajan, J. Keller, L. J. Dishaw, B. T. Furman, and M. Breitbart. Draft genome sequence of gracilimonas sp. strain bcb1 isolated from the gill tissue of the lucinid bivalve stewartia floridana in pinellas county, florida, usa. *Microbiology Resource Announcements*, 14(11):e0059525, 2025. ISSN 2576-098X. doi: 10.1128/mra.00595-25.

Y. Liu, Y. Li, E. Chen, J. Xu, W. Zhang, X. Zeng, and X. Luo. Repeat and haplotype aware error correction in nanopore sequencing reads with dechat. *Communications Biology*, 7(1): 1678, 2024. ISSN 2399-3642. doi: 10.1038/s42003-024-07376-y.

W. Liu-Wei, W. van der Toorn, P. Bohn, M. Hölzer, R. P. Smyth, and M. von Kleist. Sequencing accuracy and systematic errors of nanopore direct rna sequencing. *BMC Genomics*, 25(1):528, 2024. ISSN 1471-2164. doi: 10.1186/s12864-024-10440-w.

H. Lodhi, C. Saunders, J. Shawe-Taylor, N. Cristianini, and C. Watkins. Text classification using string kernels. *Journal of machine learning research*, 2(Feb):419–444, 2002.

A. H. Moeller, B. A. Dillard, S. L. Goldman, M. V. F. Real, and D. D. Sprockett. Removal of sequencing adapter contamination improves microbial genome databases. *BMC Genomics*, 25(1): 1033, 2024. ISSN 1471-2164. doi: 10.1186/s12864-024-10956-1.

D. H. Parks, M. Chuvochina, C. Rinke, A. J. Mussig, P.-A. Chaumeil, and P. Hugenholtz. Gtdb: an ongoing census of bacterial and archaeal diversity through a phylogenetically consistent, rank normalized and complete genome-based taxonomy. *Nucleic Acids Research*, 50(D1):D785–D794, 2021. ISSN 1362-4962. doi: 10.1093/nar/gkab776.

K. D. Pruitt, T. Tatusova, and D. R. Maglott. Ncbi reference sequences (refseq): a curated non-redundant sequence database of genomes, transcripts and proteins. *Nucleic Acids Research*, 35(Database):D61–D65, 2007. ISSN 1362-4962. doi: 10.1093/nar/gkl842.

F. J. Rang, W. P. Kloosterman, and J. de Ridder. From squiggle to basepair: computational approaches for improving nanopore sequencing read accuracy. *Genome Biology*, 19(1):90, 2018. ISSN 1474-760X. doi: 10.1186/s13059-018-1462-9.

M. Rosenfeld. Upper bounds on the average edit distance between two random strings. *ArXiv*, abs/2407.18113, 2024. doi: 10.48550/ARXIV.2407.18113.

S. J. Salter, M. J. Cox, E. M. Turek, S. T. Calus, W. O. Cookson, M. F. Moffatt, P. Turner, J. Parkhill, N. J. Loman, and A. W. Walker. Reagent and laboratory contamination can critically impact sequence-based microbiome analyses. *BMC Biology*, 12 (1):87, 2014. ISSN 1741-7007. doi: 10.1186/s12915-014-0087-z.

H. Samarakoon, K. Liyanage, J. M. Ferguson, S. Parameswaran, H. Gamaarachchi, and I. W. Deveson. Interactive visualization of nanopore sequencing signal data with squigualiser. *Bioinformatics*, 40(8):btae501, 2024. ISSN 1367-4811. doi: 10.1093/bioinformatics/btae501.

T. Sauvage, A. Cormier, and P. Delphine. A comparison of oxford nanopore library strategies for bacterial genomics. *BMC Genomics*, 24(1):627, 2023. ISSN 1471-2164. doi: 10.1186/s12864-023-09729-z.

L. Song and B. Langmead. Centrifuger: lossless compression of microbial genomes for efficient and accurate metagenomic sequence classification. *Genome Biology*, 25(1):106, 2024. ISSN 1474-760X. doi: 10.1186/s13059-024-03244-4.

M. Steinegger and J. Söding. Mmseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35(11):1026–1028, 2017. ISSN 1546-1696. doi: 10.1038/nbt.3988.

D. K. Sullivan and L. Pachter. Flexible parsing, interpretation, and editing of technical sequences with splitcode. *Bioinformatics*, 40(6):btae331, 2024. ISSN 1367-4811. doi: 10.1093/bioinformatics/btae331.

T. Tatusova, S. Ciufo, B. Fedorov, K. O'Neill, and I. Tolstoy. Refseq microbial genomes database: new representation and annotation strategy. *Nucleic Acids Research*, 42(D1): D553–D559, 2013. ISSN 1362-4962. doi: 10.1093/nar/gkt1274.

T. Weinmaier, R. Conzemius, Y. Bergman, S. Lewis, E. B. Jacobs, P. D. Tamma, A. Materna, J. Weinberger, S. Beisken, and P. J. Simner. Validation and application of long-read whole-genome sequencing for antimicrobial resistance gene detection and antimicrobial susceptibility testing. *Antimicrobial Agents and Chemotherapy*, 67(1):e0107222, 2023. ISSN 1098-6596. doi: 10.1128/aac.01072-22.

R. R. Wick, M. B. Schultz, J. Zobel, and K. E. Holt. Bandage: interactive visualization of de novo genome assemblies. *Bioinformatics*, 31(20):3350–3352, 2015. ISSN 1367-4803. doi: 10.1093/bioinformatics/btv383.

R. R. Wick, L. M. Judd, and K. E. Holt. Assembling the perfect bacterial genome using oxford nanopore and illumina sequencing. *PLOS Computational Biology*, 19(3):e1010905, 2023. ISSN 1553-7358. doi: 10.1371/journal.pcbi.1010905.

X. Wu, H. Luo, F. Xu, C. Ge, S. Li, X. Deng, M. Wiedmann, R. C. Baker, A. Stevenson, G. Zhang, and S. Tang. Evaluation of salmonella serotype prediction with multiplex nanopore sequencing. *Frontiers in Microbiology*, 12:637771, 2021. ISSN 1664-302X. doi: 10.3389/fmicb.2021.637771.

Y. Xu, K. Lewandowski, S. Lumley, S. Pullan, R. Vipond, M. Carroll, D. Foster, P. C. Matthews, T. Peto, and D. Crook. Detection of viral pathogens with multiplex nanopore minion sequencing: Be careful with cross-talk. *Frontiers in Microbiology*, 9:2225, 2018. ISSN 1664-302X. doi: 10.3389/fmicb.2018.02225.

A. Further methods

A.1. Rapid barcoding patterns

For rapid barcoding we will consider the following two patterns safe:

1. `Ftag[fw,*,@left(0..250),>>]`
2. `Ftag[fw,?1,@left(0..250)]__Ftag[fw,?1,@prev_left(0..250),>>]`

The first pattern is the ideal pattern, with just a single left tag. The second pattern covers the second most common pattern (later in Results), in which two barcodes are ligated, and we take the label from the inner tag (the one with `>>`), although we do enforce that both barcodes are the same using the `?1` wildcard.

In case we want to maximise (`--maximize`) matches potentially at the expense of accuracy, we add the following patterns:

1. `Ftag[fw,*,@left(0..250)]__Ftag[fw,*,@prev_left(0..250),>>]`
2. `Ftag[fw,*,@left(0..250),>>]__Ftag[<<,fw,*,@right(0..250)]`
3. `Ftag[fw,*,@left(0..250)]__Ftag[fw,*,@prev_left(0..250),>>]__Ftag[<<,fw,*,@right(0..250)]`

Here we always determine the sample based on the inner barcode, but are more flexible by allowing additional barcodes to be present. Using maximise patterns will give the highest yield and should be used for tasks such as assembly. However, for diagnostics and quantification, where false positives may affect the outcome, it might be better to use just the safe patterns.

A.2. DNA isolation and sequencing

66 unidentified bacterial and fungal isolates were selected for Nanopore sequencing for diagnostic purposes (Table S1). Briefly, genomic DNA was isolated using the DNeasy Ultra Clean Microbial kit (Qiagen, Venlo, the Netherlands). Nanopore sequencing was performed according to the rapid barcoding protocol RBK96.114 on an R10.4.1 flow cell with MinION (Oxford Nanopore, Oxford, UK). Bases were called using super accurate basecalling using MinKNOW v24.11.10.

A.3. Sequence searching and assemblies

For the analyses, we used Sassy [Beeloo and Groot Koerkamp, 2026a] to search based on edit distance, always using an overhang of $\alpha = 0.5$ (`-a 0.5`) to find matches crossing read boundaries and the IUPAC alphabet to handle ambiguous bases (`--alphabet iupac`). To identify rapid barcoding contamination, we require that—aside from the flanks—a barcode is detected within ≤ 4 edits. Rapid barcoding kits use the *Mu* transposase for barcode and adapter attachment. Since the *Mu* transposase is naturally encoded by the *Mu* phage, which infects *Enterobacteriaceae*, searching for just the flank could produce false positive matches—cases where the match reflects the presence of *Mu* phage rather than true contamination. Annotation of our genomes showed two *Enterobacteriaceae* species in our own dataset. Moreover, when we later searched databases, these included many *Enterobacteriaceae* species.

As noted previously (Section 2.1), a typical rapid barcode flank consists of a left flank (F_ℓ), a barcode (B), and a right flank (F_r). In experimental data, however, we observed reads with two barcodes on the left side, following this concatenation configuration:

$$F_\ell \circ B \circ F_r \circ B \circ F_r.$$

In this case, the second barcode entirely lacks its left flank and instead appears directly adjacent to the right flank of the preceding barcode region. We refer to such structures as fusions, and we searched for them in our datasets using the pattern `GTTTTTCGTGCGCCGCTTCA<barcode_seq>GTTTTCGCATTTATCGTGAAACG`. To detect fusions, we used MMseqs2 [Steinegger and Söding, 2017] using the following parameters: `-search-type 3`, `-max-seqs 5000000`, `-max-seq-len 200000`. We initially used MMseqs2 instead of Sassy, since it was unclear whether fusion events would appear primarily as semi-global matches or also as shorter local sub-matches.

Typically, Filtlong (`https://github.com/rrwick/filtlong`) is used to discard the worst 10% of reads prior to assembly. Since this depends on how well the demultiplexer has already removed low-quality reads, we instead applied absolute thresholds. Reads were filtered with Filtlong (`v0.2.1`) keeping those $\geq$ 1000 bp (`-min_length 1000`) and with mean quality $\geq$ 15 (`-min_mean_q 15`). To assemble the genomes we used Flye (`v2.9.6-b1802`) [Kolmogorov et al., 2019] in `-ont-hq` mode with 5 polishing iterations (`-i 5`), followed by a final polishing using Medaka (`https://github.com/nanoporetech/medaka`). To map sequences to assemblies we used minimap2 (`v2.28-r1209`) [Li, 2018], in `map-ont` mode (default parameters). To compare assemblies, we first extracted the contigs using Samtools [Li et al., 2009], then mapped these to each other using Minimap2 (`map-ont`) [Li, 2018], followed by graph induction using seqwish [Garrison and Guarracino, 2022] and visualised using Bandage [Wick et al., 2015].

A.4. Taxonomic annotation

We used Centrifuger [Song and Langmead, 2024] with the RefSeq database [Pruitt et al., 2007] and Genome Taxonomy Database (GTDB) (`r226`) [Parks et al., 2021]. As the GTDB alone does not include fungal sequences, we used the pre-built GTDB + fungi database provided by Centrifuger. To link taxonomy identifiers to taxonomic lineages we used ete3 [Huerta-Cepas et al., 2016].

B. Subsequence barcode scoring scheme

Motivation. In Nanopore sequencing, errors often occur as localised stretches of incorrect or missing nucleotides, typically caused by slippage or transient stalling of DNA in the pore [Delahaye and Nicolas, 2021]. A single error stretch (e.g. TTTT) can already introduce multiple consecutive edits in an otherwise correct alignment. In contrast, observing the same number of edits scattered across an entire barcode is much less likely to result from such localised effects. Consequently, a scoring scheme that rewards compact groups of matches provides improved discrimination between true barcode matches and alignments dominated by localised error runs (see Figure S1).

Setup. Given an edit-distance alignment between a barcode and a read, we consider its CIGAR string C. From C we extract the query positions that are exact matches (i.e. not substitutions, insertions, or deletions), and index them in increasing order as

$$P = (p_1 < p_2 < \cdots < p_{|P|}).$$

Let $k \geq 1$ be a subsequence length and let $\lambda \in (0, 1]$ be a decay parameter.

Score definition. The subsequence score $S_k(C;\lambda)$ counts all increasing k-tuples of match positions and weights each tuple by an exponential penalty that depends on its span. Formally,

$$S_k(C;\lambda) \;=\; \sum_{1\le i_1<i_2<\cdots<i_k\le|P|} \lambda^{\,p_{i_k}-p_{i_1}+1}.$$

If $|P|<k$, we define $S_k=0$. Smaller values of λ penalise widely spaced matches more strongly, whereas values of λ close to 1 treat spacing more uniformly. Intuitively, compact runs of k matches contribute substantially more to the score than the same number of matches distributed across a larger region of the barcode.

Efficient evaluation. Following the subsequence kernel formulation of Lodhi et al. [2002], we compute S_k using dynamic programming on the match positions. Let $D_i^{(t)}$ denote the total weight of all increasing t-tuples whose last element is p_i. Then

$$D_i^{(1)} = \lambda,$$

$$D_i^{(t)} = \sum_{j<i} D_j^{(t-1)}\,\lambda^{\,p_i-p_j}, \qquad t=2,\dots,k.$$

The final score is obtained as $S_k=\sum_i D_i^{(k)}$. In practice, $|P|$ is bounded by the barcode length, making this computation feasible for typical parameter choices. While this approach could arguably be used to directly search both flanks and barcodes, it is prohibitively slow compared to edit distance, which can be accelerated using SIMD instructions; see Beeloo and Groot Koerkamp [2026a,b] for more details. An implementation operating directly on CIGAR strings is available in our crate.[5]

Usage. In this work, edit distance is used to locate τ_N within a read, while subsequence scoring is applied in a second stage to discriminate between candidate barcodes. This separation allows us to retain computational efficiency during localisation while exploiting the increased sensitivity of subsequence scoring during barcode assignment.

Full calculation of CIGAR examples.

We illustrate the score for $k=3$ with decay λ on two CIGAR strings. Given match positions $\textsc{pos}=(p_1<\cdots<p_{|}C|)$, the score is

$$K_3(C;\lambda) \;=\; \sum_{1\le i<j<\ell\le|C|} \lambda^{\,p_\ell-p_i+1}.$$

There is exactly one triple when $|C|=3$, namely $(i,j,\ell)=(1,2,3)$, so $K_3=\lambda^{\,p_3-p_1+1}$.

Example 1: `Sub Sub Match Match Match.` (Figure S1A) Advancing the alignment index by each operation yields match positions $\textsc{pos}=(2,3,4)$. The only 3-subsequence is $(2,3,4)$ with inclusive span $4-2+1=3$, hence

$$K_3 \;=\; \lambda^3.$$

For $\lambda=\frac{1}{2}$, $K_3=2^{-3}=\frac{1}{8}=0.125$.

Example 2: `Match Sub Match Sub Match.` (Figure S1B) Match positions are $\textsc{pos}=(0,2,4)$. The only 3-subsequence is $(0,2,4)$ with inclusive span $4-0+1=5$, hence

$$K_3 \;=\; \lambda^5.$$

For $\lambda=\frac{1}{2}$, $K_3=2^{-5}=\frac{1}{32}=0.03125$.

[5] `https://github.com/rickbeeloo/cigar-lodhi-rs.git`

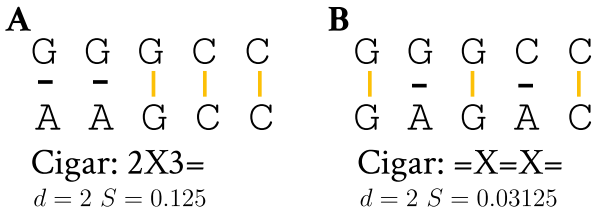


Fig. S1: Example of edit distance and subsequence scoring. Both alignments here have the same edit distance of 2, but in **(A)** the matches and errors are continuous, whereas in **(B)** the matches and errors are interleaved. Considering Nanopore errors arising from slippage and stalling of DNA in the pore, the alignment in **(A)** is more likely to be correct, which is captured by the subsequence scoring (S).

Interpretation. Both examples contain exactly one ordered triple of matches; the difference is the spacing between the first and last matches. The second CIGAR has larger gaps (due to substitutions), increasing the span and thus down-weighting the contribution more strongly via the λ^{span} factor.

Parameter choice. Throughout this work we use $k=3$ and $\lambda=0.5$. These values were selected heuristically to capture short, locally consistent runs of matches, which are characteristic of correct Nanopore alignments in the presence of localised error bursts (see Examples 1 and 2 above, where these parameters yield a 4× score difference between compact and dispersed alignments of identical edit distance). Since the score is used only for relative ranking of candidate barcodes, the exact values of k and λ are not critical: for any $\lambda\in(0,1)$ and $k\ge 2$, a compact alignment will always score higher than a dispersed one with the same number of matches, preserving the ranking regardless of the specific parameter choice. We therefore did not perform fine-grained parameter tuning.

C. Cut-offs for flank and barcode matching

A key step in demultiplexing is determining thresholds that decide whether a region matching the flank τ_N and barcode B should be accepted as a true match. As in other tools, Barbell uses edit distance to locate τ_N, but employs subsequence scoring for barcode discrimination. Because both measures depend on sequence length and scoring parameters, cut-offs are derived automatically and reported to the user.

Edit-distance cut-off for flank localisation.. The expected edit distance between two random strings is approximately 51% of their length, with observed values typically ranging between 36% and 63% [Rosenfeld, 2024]. In rapid barcoding, the flank τ_N has length $|\tau_N|=90$, but contains an internal mask $N_{|B|}$ of 24 N characters that match any base. We therefore define the effective flank length as

$$|\tau_N|_{\text{eff}} = |\tau_N| - |B| = 66.$$

Using the theoretical lower bound, one would expect approximately $66\cdot 0.36\approx 24$ edits when aligning a random string to the effective flank. However, these bounds are derived from simulations on long sequences and are overly permissive for shorter tags. To address this, we empirically fitted a lower bound to edit-distance distributions obtained from random short strings (see

Figure S3), yielding

$$\theta_{\mathrm{emp}}(a) = \max\bigl(0,\ \lceil 0.51 \cdot a - 1.7312 \cdot \sqrt{a} \rceil\bigr)\,.$$

Here, $\theta_{\mathrm{emp}}(a)$ denotes the maximum allowed edit distance for a sequence of length a, and the subscript "emp" indicates that this bound is empirically derived. The $\sqrt{a}$ term adjusts the theoretical 51% error rate downward for shorter sequences, imposing a stricter cut-off that gradually relaxes as sequence length increases.

Applying this formula to the effective flank length gives

$$\theta_{\mathrm{emp}}(66) = 20.$$

We verified that using the theoretical value of 24 edits results in false positive flank matches within reads, whereas the empirically derived cut-off of 20 avoids these cases. Barbell therefore uses $\theta_{\mathrm{emp}}(a)$ to set the flank edit-distance cut-off automatically based on the length of the user-provided tags. The selected cut-off is reported to the user and can be manually adjusted if more or less stringent matching is desired (Section 2.5).

Cut-offs for barcode assignment using subsequence scores. Existing tools such as Dorado and Flexiplex use edit-distance-based heuristics to assign barcodes after locating the flanks. Flexiplex enforces a maximum edit distance (6 in this manuscript) and discards reads when multiple barcodes share the same cost. Dorado applies a more complex heuristic that restricts barcode searches to expected locations and requires the top-scoring barcode to be sufficiently separated from the second-best hit.[6]

In contrast, Barbell scores barcode regions using the subsequence scoring scheme described in Appendix B. Because the subsequence score $S_k(C;\lambda)$ depends on the parameters k and λ, its absolute value is not directly comparable across settings. To make score thresholds interpretable, we normalise scores relative to the maximum achievable score for a perfect alignment. Specifically, we compute the perfect score corresponding to a CIGAR string consisting entirely of matches. For barcodes of length 24, this corresponds to a CIGAR string of `24=`. Barcode matches are then accepted if their score exceeds a user-defined fraction $S_{\min}$ of this perfect score, and if the difference between the best and second-best barcode scores exceeds an absolute fraction S_{diff}. By default, we use $S_{\min} = 0.2$ and $S_{\mathrm{diff}} = 0.1$. To give an intuition for how these values affect the number of demultiplexed reads, and the percentage of correct reads, we show a grid in Figure S2. This normalisation allows users to reason about barcode confidence in intuitive percentage terms, while retaining the error sensitivity of subsequence scoring.

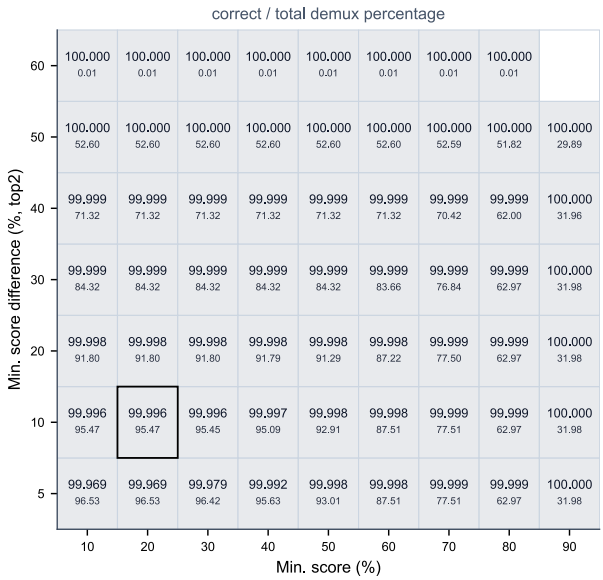


Fig. S2: $S_{\min}$ **score versus** S_{diff} **grid**. We first select 788,672 reads from our data where we had long hits to taxonomy entries using Centrifuger (min. trimmed length 250 bp) or barcode matches with edits unlikely due to chance (max 4 edits). Note that *trimmed* here is not based on any tool to avoid biases, instead we removed the first 200 bp of each read to certainly remove rapid adapters/barcodes ($\approx$ 90 bp). Then we ran Barbell on these reads varying the minimum score ($S_{\min}$; x-axis) and the minimum score difference between the top 2 matches (S_{diff}; y-axis). The former being the minimum subsequence score to assign the barcode, and the latter the difference between the top two best scoring barcodes (Algorithm 1). The upper value in each cell is the percentage "correct" (i.e. correct barcode based on taxonomy or low edit distance), and the bottom is the percentage of the total reads demultiplexed. The default cut-off is shown in the bold-bordered cell. Note that the remainder, total - correct, does not necessarily indicate an "incorrect" demultiplexing, as some reads receive unexpected barcodes (e.g. after pooling), could not be taxonomically annotated, or had high edit distances. Nevertheless, this grid provides a useful overview of how much change can be expected when varying either cut-off.

C.1. Comparison of assembly contiguity between Dorado and Barbell

Assembly contiguity for Barbell- and Dorado-trimmed reads was assessed using QUAST-style contiguity profiles. In general, assembly contiguity can be summarised using NGx curves (and corresponding area-under-the-curve metrics such as auNG), which report contig length as a function of genome coverage percentile and require knowledge of the expected or reference genome size. Because an expected genome size was not available for these samples, true NGx curves could not be computed.

Instead, we constructed cumulative contig length–rank curves by ordering contigs from longest to shortest and plotting cumulative assembled sequence length as a function of contig rank. Each curve was normalised by the total assembly length for that barcode so that the y-axis represents the fraction of the assembly accumulated. To focus on biologically meaningful contiguity and to reduce sensitivity to very small contigs, we summarised each curve by the area under the curve up to 95% of the assembled sequence ($\mathrm{AUC}_{0.95}$).

Because the x-axis is contig rank, $\mathrm{AUC}_{0.95}$ has units of contigs and can be interpreted as the average number of contigs required to recover the first 95% of the assembly. Differences in $\mathrm{AUC}_{0.95}$ therefore reflect shifts in how rapidly assembled sequence accumulates into long contigs, conditional on the assembled sequence, rather than differences in total assembly size or completeness. Higher $\mathrm{AUC}_{0.95}$ values indicate assemblies in which more contigs are required to reach the same fraction of the assembly, consistent with greater fragmentation.

[6] As there is no dedicated publication for Dorado, see the source files `barcode_kits.h` and `BarcodeClassifier.cpp` in the Dorado GitHub repository.

Barcode	Species	Barcode	Species
barcode01	*W11650 sp030535295*	barcode34	*Cutibacterium acnes*
barcode04	*Malassezia restricta*	barcode35	*Fusobacterium russii*
barcode05	*Castellaniella denitrificans*	barcode36	*Mycobacterium abscessus*
barcode06	*Psychrobacter sanguinis*	barcode37	*Yamadazyma tenuis*
barcode07	*JAUMYT01 sp030528525*	barcode38	*Mycobacterium smegmatis*
barcode08	*QD2021 sp036209505*	barcode40	*Clostridium sp036643715*
barcode09	*Exiguobacterium_A sp038006045*	barcode41	*QD2021 sp036209505*
barcode10	*Muribacter muris*	barcode42	*Actinobacillus_C sp020026155*
barcode11	*Acinetobacter terrestris*	barcode43	*Mannheimia granulomatis*
barcode12	*Pasteurella felis*	barcode44	*Yersinia pestis*
barcode13	*Prescottella sp032085135*	barcode45	*Chelonobacter testudinis*
barcode14	*Psychrobacter sanguinis*	barcode46	*Brucella melitensis*
barcode15	*Acinetobacter sp947627655*	barcode47	*Actinobacillus_C sp020026155*
barcode16	*Capnocytophaga catalasegens*	barcode48	*Micrococcus luteus*
barcode18	*Frederiksenia canicola*	barcode49	*Saccharomyces cerevisiae*
barcode19	*Granulicatella balaenopterae*	barcode50	*Rodentibacter trehalosifermentans*
barcode20	*Granulicatella balaenopterae*	barcode51	*Actinomyces denticolens*
barcode22	*Planococcus glaciei*	barcode52	*Brevibacterium gallinarum*
barcode23	*Fastidiosipila sp963510375*	barcode53	*Streptococcus equi*
barcode24	*Brachybacterium conglomeratum*	barcode54	*Mannheimia haemolytica*
barcode25	*Prescottella equi*	barcode56	*Buchananella hordeovulneris*
barcode26	*Prescottella equi*	barcode57	*Staphylococcus simulans_B*
barcode27	*Mannheimia haemolytica*	barcode59	*QD2021 sp036209505*
barcode28	*Carnobacterium maltaromaticum*	barcode60	*Bisgaardia hudsonensis*
barcode29	*Nicoletella semolina*	barcode61	*Burkholderia thailandensis*
barcode30	*Capnocytophaga stomatis*	barcode62	*Intestinirhabdus alba*
barcode31	*Gordonia sp016919385*	barcode63	*Actinomyces denticolens*
barcode32	*Berryella intestinalis*	barcode64	*Streptococcus pasteurianus*
barcode33	*Berryella intestinalis*	barcode65	*Streptococcus gallolyticus*
		barcode66	*Streptococcus pasteurianus*

Table S1. List of barcodes and their corresponding species assignments based on Centrifuger (GTDB+Fungi), sorted by barcode. barcode02 and barcode03 are not included in the table due to insufficient reads for assembly, which prevented taxonomic annotation. We note that barcodes barcode44 and barcode46 are likely incorrectly annotated by Centrifuger and should be *Yersinia pseudotuberculosis* and *Brucella ceti*, respectively, based on more extensive analysis using an in-house pipeline. Names like *JAUMYT01* and *QD2021* are GTDB placeholders and full taxonomic information is available at GTDB e.g. `https://gtdb.ecogenomic.org/searches?s=al&q=QD2021`.

For each barcode, differences in contiguity between Dorado- and Barbell-based assemblies were calculated as $\Delta\mathrm{AUC}_{0.95} = \mathrm{AUC}_{0.95,\mathrm{Dorado}} - \mathrm{AUC}_{0.95,\mathrm{Barbell}}$. Across the 64 shared barcodes, $\Delta\mathrm{AUC}_{0.95}$ values were centred at zero (median $\Delta\mathrm{AUC}_{0.95} = 0$; mean -0.078; bootstrap 95% confidence interval for the median: $[0, 0]$), indicating no systematic difference in contiguity between tools (Supplementary Fig. S4). Although individual barcodes showed larger positive or negative differences, these deviations were balanced in both directions and did not result in a significant overall shift in contiguity (paired Wilcoxon signed-rank test; $p = 0.22$).

D. Mu phage integration

To investigate the source of ambiguous *Enterobacteriaceae* assignments among trimmed Dorado reads, we examined whether residual *Mu*-transposon sequences from the Nanopore Rapid Barcode flanks might have matched endogenous *Mu* transposons in these bacteria. The 26,763 Dorado-trimmed reads were aligned to the 244 RefSeq genomes, and the genomic regions within 5 kb of the alignment sites were analysed. Most alignments (24,857 reads; 92.9%; see Section 3.2) were located near genes characteristic of transposons and nearby cargo, including *tail fibre assembly protein* (24,154 reads), *recombinase family protein* (24,153), *Mom family adenine methylcarbamoylation protein* (24,152), and *tail fibre protein* (24,152).

E. Double barcode attachment and bleeding

As reported in Section 3.2, 213,016 reads (4.3%) carried two left-end barcodes, of which 100,042 (47.0%) showed a direct barcode fusion with higher mean edit distance to the second barcode than to the first (7 vs. 3; Figure S6). Fusions were observed across all barcodes, but the prevalence of fusion-associated deletions in the first 6 bp of the second barcode varied by barcode, for example: BC05, 95.4% (2,073/2,173); BC25, 95.8% (2,106/2,198); BC61, 29.5% (901/3,053); and BC45, 52.3% (2,027/3,874). We observed similar patterns when analysing public datasets (Weinmaier et al.

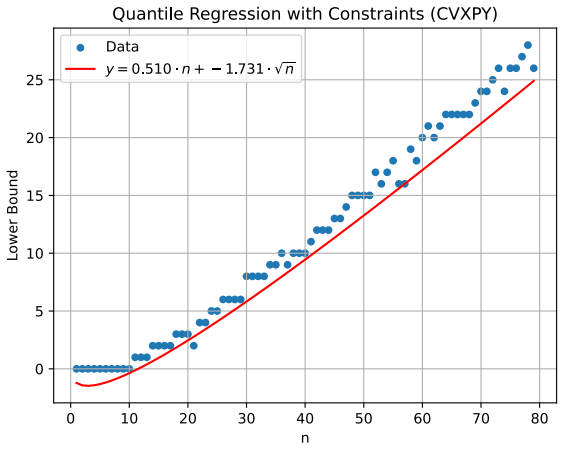


Fig. S3: Lower bound edit distance fit. For each length (x-axis) we performed 1 million (uniform) random DNA versus (uniform) random DNA comparisons, and plotted the lowest observed edit distance (lower bound, y-axis). We then fitted a line through the lower 1% quantile, resulting in $\theta_{\text{emp}}(a) = \max\left(0,\ \lceil 0.51 \cdot a - 1.7312 \cdot \sqrt{a} \rceil\right)$. The fit was obtained by solving a constrained optimisation problem (minimising the fitting error while enforcing non-negativity) using the Python package cvxpy. Naturally, with more data, the chance of hits in reads increases, and choosing cut-offs will remain a balance between assigned reads and false positives. Nevertheless, the cut-offs from this fit have been shown to work well in practice[7]

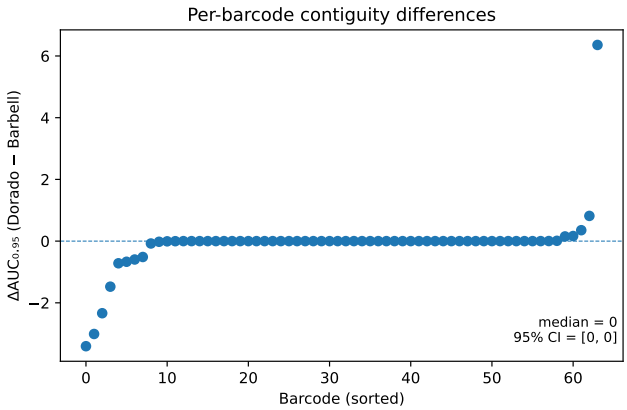


Fig. S4: Per-barcode differences in assembly contiguity between assemblies from Dorado and Barbell trimmed reads. For each assembly, contigs were ordered from longest to shortest and the cumulative assembled length was plotted as a function of contig rank, then normalised by the total assembly length. The area under this curve up to 95% of the assembly (AUC0.95) provides a quantitative summary of assembly contiguity, reflecting how many contigs are required to recover most of the assembly: more contiguous assemblies reach 95% with fewer contigs and therefore have lower AUC0.95 values. Differences between methods are shown as $\Delta\text{AUC}0.95 = \text{AUC}0.95_{\text{Dorado}} - \text{AUC}0.95_{\text{Barbell}}$ across 64 shared barcodes. Positive values indicate that Dorado assemblies require more contigs to reach 95% of the assembled sequence (i.e., are more fragmented), whereas negative values indicate greater fragmentation in Barbell assemblies. This metric captures differences in contig length distributions (assembly fragmentation) and does not reflect base-level sequence accuracy. The dashed line denotes $\Delta\text{AUC}0.95 = 0$. The median difference was 0 (bootstrap 95% CI: [0, 0]), indicating similar assembly contiguity between the two methods.

[2023]: BC05, 372/393, 94.7%; Di Pilato et al. [2025]: BC45, 61/202, 30.2%). Scanning all untrimmed reads for the fusion pattern revealed that 3.3% of all reads (165,396) contained such a double-barcode fusion.

In the 968 reads where the two barcodes differed (Section 3.2), Dorado consistently reported the outer copy and Barbell the inner copy. To evaluate which assignment was correct, we compared read-level taxonomic annotations with those of the assemblies linked to the assigned barcodes. This approach is limited by the uncertainty of read-level annotations (here filtered at $\geq$ 100 bp hit length; see Section 3.2) and by the fact that nine species were present in two samples (Table S1). Taxonomic annotation supported the inner barcode in 515 cases (53.2%) versus 43 (4.4%) for the outer, with the remainder being unclassified.

F. Pore signal examples

To study the pore signal we used the raw pod5 files and basecalled these using Dorado's super-accurate model, emitting the move table (`--emit-moves`). We then converted the pod5 files to slow5 using blue-crab [Gamaarachchi et al., 2022], and visualised the pore signals and basecalled reads using Squigualiser [Samarakoon et al., 2024], see Figure S8. In case of secondary structure formation we expected the pore signal intensity (y-axis) to increase drastically (e.g. double). However, this was not the case.

G. Analysis of SRA datasets

To investigate the barcode pattern in datasets other than the one sequenced by us, we queried the SRA database for the terms "SQK-RBK114" and "SQK-NBD114" and selected 8 random datasets for each (including our own for rapid). These datasets along with the identifiers used in Figure 4 are listed in Table S2.

Read-level re-analyses and trimming

This appendix documents the read-level re-analyses used to validate and repair assemblies that contained barcode or flank sequences. Full command lines, parameters, and logs are provided here for reproducibility.

Photobacterium leiognathi (`CP131573.1`). The uploaded contigs were 3,089,627 bp and 1,431,556 bp. We re-downloaded 89,663 raw reads from SRA. Pattern counts and trimming summary:

- 73,580 reads (82%) contained the expected single-flank pattern `Ftag[fw,*,@left(0..250)]`.
- 1,299 reads (1.4%) contained a double-flank arrangement `Ftag[fw,*,@left(0..250)]_Ftag[fw,*,@prev_left(0..250)]`.

Trimming with Barbell (default options for SQK-RBK114-96) and reassembly produced three circular contigs (3,176,913 bp; 1,497,394 bp; 15,997 bp) and a small linear contig (4,109 bp). The 1.49 Mb contig matched `CP131573.1` without barcode sequences.

E. coli plasmid (`CP165501.1`). We re-downloaded 113,399 raw reads from SRA and demultiplexed with Barbell (default; SQK-RBK114-96):

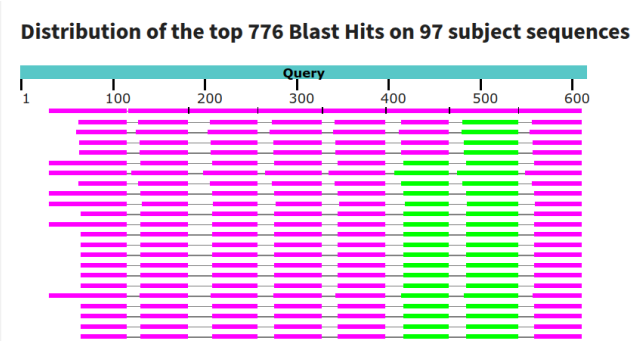


Fig. S5: Artefact read with 8 barcodes. This figure shows the BLAST output for read `52369018-4a3c-433b-881b-e46226500fb6` (611 bp) against all possible rapid barcode flanks and barcodes. The read consists entirely of barcode and flank sequences. Barbell detected 8× an `Ftag` in this read. The "Query" represents the read sequence. Each bar corresponds to a BLAST hit: pink bars indicate alignment scores of 80–200, and green bars 50–80. Because one region can match multiple barcode or flank sequences, matches appear underneath each other, with the highest-scoring ones shown on top. As expected, we observe eight distinct blocks (or "columns"), matching the number of `Ftag`s detected by Barbell.

Table S2. SRA run accessions corresponding to labels D1–D16 in Figure 4.

Label	SRA accession
D1	SRR36128788
D2	SRR36347832
D3	SRR36496803
D4	SRR36760490
D5	SRR36804021
D6	SRR36844237
D7	SRR37153304
D8	SRR37504295
D9	ERR15726106[a]
D10	SRR36318375
D11	SRR36516319
D12	SRR36688800
D13	SRR36911183
D14	SRR37228286
D15	SRR37630668
D16	SRR37857610

[a] Our dataset.

- 97,461 reads (86%) contained the expected single-flank pattern.
- According to Barbell, 76.4% of reads were assigned to BC09 and 13.1% to BC10.

Assembling reads trimmed and binned for BC09 produced a 43,165 bp contig matching `CP165501.1` from positions 111 to 39,256, consistent with removal of barcode overhangs at both contig ends.

Notes and reproducibility. All trimming and assembly commands, parameter files, and run logs (including the exact

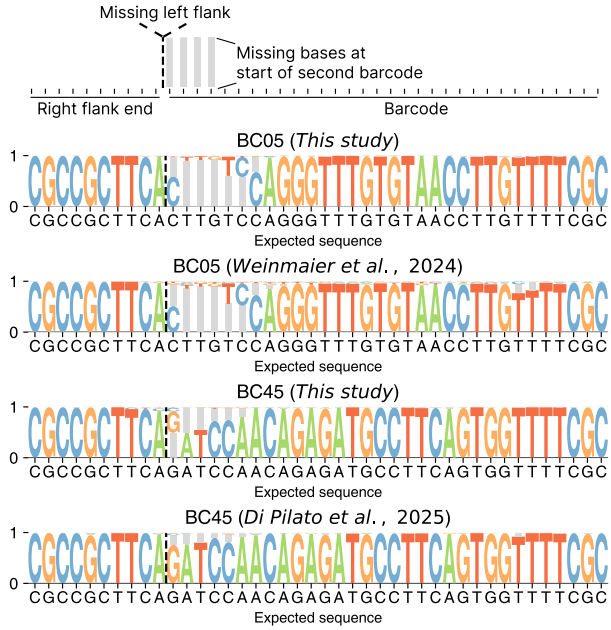


Fig. S6: Detection of fused rapid barcodes and associated deletions. Among 213,016 reads with two left barcodes, 100,042 showed an unusual arrangement in which the right flank adjacent to the first barcode was directly fused to the second barcode (see Appendix A.3). The sequence logos show the fusion junctions for BC05 and BC45 in our and public datasets. Letter height indicates base frequency; grey bars mark alignment gaps (missing bases). In typical reads, a left flank, barcode, and right flank are observed in order, whereas fusion reads show the right flank of the first barcode (ending with TTCA, dashed line) joined directly to the second barcode, always lacking its left flank (not shown) and first part of the second barcode. Deletions within the first 6 bp of the second barcode occurred in 95.4% (BC05) and 52.3% (BC45) of our reads, and with comparable frequencies in public datasets (94.7% and 30.2%, respectively). Loss of the first 1–6 bp of the second barcode was thus more frequent in fusions involving BC05 than BC45.

Barbell and assembler versions used) are archived with the Zenodo record cited in the main text, and code can be found at Zenodo[8].

[8] `https://doi.org/10.5281/zenodo.19494282`

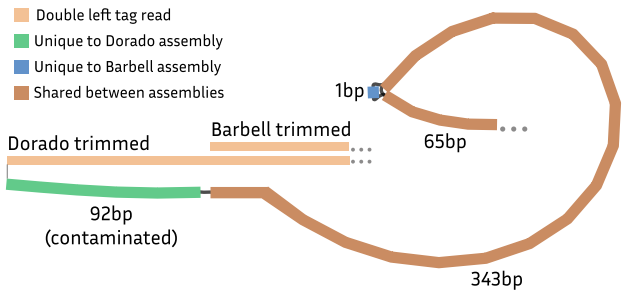


Fig. S7: Merged assembly graphs for a *Saccharomyces cerevisiae* contig. Genome assembly of a single contig from *Saccharomyces cerevisiae* using Dorado- and Barbell-trimmed reads (23,203 bp vs. 23,050 bp). Shown are the first 500 bp of the Dorado assembly and the corresponding region from the Barbell assembly. The nodes represent unitigs, and the edges their connections. The assemblies were identical except for the first 92 bp and a single nucleotide difference. The extra 92 bp in the Dorado assembly originated from a single barcode sequence left untrimmed by Dorado. The difference was caused by one read containing two left barcodes; Dorado removed only the outer barcode and left the inner one intact, since it was incorporated into the assembly. In contrast, Barbell removed both barcodes, preventing this contamination.

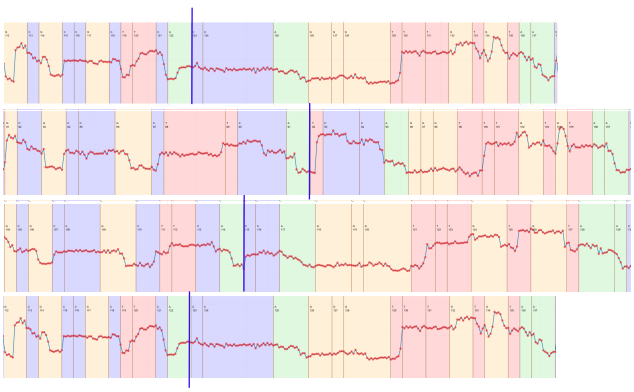


Fig. S8: Examples of the pore signal (line with red dots) for BC05 reads. The vertical blue line indicates the fusion point between the end of the right flank (`..GCTTCA`) and the beginning of the partial BC05 barcode (`CTTGTCCAGGGTTTGTGTAACCTT`). The colours indicate the basecalled bases, G=yellow, C=blue, T=red, A=green. We did not observe abnormally long stretches of signal without any basecalled bases.

Alg. 1: Pseudocode for Barbell's `annotate` step.

Require: β: set of barcode strings; τ_N: tag with masked barcode; R: read sequence
Ensure: T': set of collapsed tag calls

▷ — *Parameters* — ◁
$S_{\text{min}} \leftarrow 0.2$ ▷ *Minimum acceptable normalised score* ◁
$S_{\text{diff}} \leftarrow 0.1$ ▷ *Minimum gap between top and second score* ◁
$W \leftarrow 5$ ▷ *Extra padding around barcode window* ◁
$S_{\text{perfect}} \leftarrow S_3(\texttt{"24="}, 0.5)$ ▷ *Ideal perfect score* ◁
$\theta_B \leftarrow 20$ ▷ *Fixed edit distance cutoff for barcodes* ◁

▷ — *Stage 1: Flank detection* — ◁
$\theta_\tau \leftarrow \max\left(0, \left\lceil 0.5100 \cdot (\tau_N - |B|) - 1.7312 \cdot \sqrt{\tau_N - |B|} \right\rceil\right)$
$M_\tau \leftarrow \text{Sassy}(\tau_N, R, \theta_\tau)$
$T \leftarrow [\,]$
for $m \in M_\tau$ **do**
  $start \leftarrow m.start$
  $strand = m.strand$
  $mask_{start} \leftarrow \max(0,\ start + |F_\ell| - W)$
  $mask_{end} \leftarrow \min(|R|,\ start + |F_\ell| + |B| + W)$
  ▷ *Note that* $R[i \ldots j] := r_i \ldots r_{j-1}$ *denotes a right-exclusive substring of* R. ◁
  $mask \leftarrow R[mask_{start} \ldots mask_{end}]$

  ▷ — *Stage 2: Barcode matching* — ◁
  $matches \leftarrow [\,]$
  $scores \leftarrow [\,]$
  for $B \in \beta$ **do**
    $M_B \leftarrow \text{Sassy}(B, mask, \theta_B)$
    for $m_b \in M_B$ **do**
      if $strand \neq m_b.strand$ **then**
        continue
      $S_{\text{abs}} \leftarrow S_3(m_b.cigar, 0.5)$
      $S_{\text{rel}} \leftarrow S_{\text{abs}} / S_{\text{perfect}}$
      append$(matches, m_b)$
      append$(scores, (S_{\text{rel}}, |matches|))$

  ▷ — *Stage 3: Candidate selection* — ◁
  if $scores \neq \emptyset$ **then**
    $scores \leftarrow \text{sort_desc}(scores)$
    $(s_1, i_1) \leftarrow scores[0]$
    $s_2 \leftarrow \begin{cases} scores[1].S_{\text{rel}}, & |scores| > 1 \\ 0, & \text{otherwise} \end{cases}$
    $\Delta \leftarrow s_1 - s_2$
    if $s_1 \geq S_{\text{min}} \wedge \Delta \geq S_{\text{diff}}$ **then**
      append$(T, matches[i_1])$ ▷ *confident barcode* `Ftag` ◁
    else
      append(T, m) ▷ *ambiguous → store flank (*`Fflank`*)* ◁
  else
    append(T, m) ▷ *no barcode → store flank (*`Fflank`*)* ◁

▷ — *Stage 4: Post-processing* — ◁
▷ *If overlap (smaller vs. larger match region) bigger than 70%, collapse, prioritising* `Ftag` *over* `Fflank` ◁
$T' \leftarrow \text{collapse}(T)$
return T'